QuantSim — Requirements (Functional & Non-Functional)

Document	Requirements Specification (FR / NFR)
System	QuantSim — AI-Assisted Market Simulation & Portfolio Risk Analytics Platform
Status	**Proposed — for team review.** Nothing here is frozen; this is the recommended baseline. Aligned to the settled architectural style (microservices, ARC §0 A1, §3).
Version	0.2 (draft) — supersedes 0.1; non-functional requirements and trace anchors realigned from a modular monolith to **microservices**. Revised after an audit against the approved seven-module proposal: source authority fixed (§0), four requirements added to close coverage gaps (FR-MKT-011, FR-RSK-018, FR-DSH-008, FR-DSH-009)
Date	2026-08-23
Companion	*QuantSim — Architecture & Physical Design* (referred to below as **ARC**)

0. Purpose, scope & conventions

This document specifies **what QuantSim must do** (functional requirements, §2) and **how well it must do it** (non-functional requirements, §3). It is the companion to the architecture document; where a requirement is realized by a specific design mechanism, its **Trace** column cites the ARC section (e.g. "ARC §8.5") so the two documents stay in lock-step. The functional requirements are organized under the **seven modules named in the approved project proposal** (M1–M7), each of which is built as one independently deployable service, plus a cross-cutting **Platform/Identity** group (M0) — eight services in the architecture (ARC §0 A1, §5). The M0–M7 labels are retained from version 0.1 so existing trace anchors continue to resolve; each label now names a service rather than a module inside one application. The mapping is made explicit in §4.

Source authority — what binds this document and what does not. Two inputs exist, and they do not carry equal weight.

The **approved project proposal** — the seven-module specification of QuantSim — is the **single source of truth for scope**. Every capability it names is carried by at least one requirement here (§4.1), the module numbering follows it exactly, and nothing it asks for is dropped, renamed away, or quietly deferred. Where the proposal is terse, a requirement makes it testable; where it is silent on a quality attribute, §3 supplies one. What a requirement may not do is contradict it.

The team's **internal brainstorm** (working notes on structure, tables, and candidate requirements) is a **non-binding input**. It has been mined for useful detail and much of that detail survives here on its own merits — the four-role access model, the alerting loop, the model registry, the point-in-time feature store, the audit trail. But it carries no design authority: where it proposed a structure, that structure was adopted only if it is the efficient answer for this system, and rejected otherwise. Three of its structural positions are rejected

outright and the rejections are recorded rather than glossed: its single six-layer stack in which "each layer talks only to the layer below" (ARC §4 — layering now lives *inside* each service, and the system level is a service topology, not a stack); its placement of every authorization decision at the API layer (ARC §9.2 — authentication happens once at the edge, the authorization decision belongs to the service that owns the data, because a gateway-only check makes every service implicitly trust whatever reaches it); and its grouping of the instrument master and basket definitions under "Authentication" (they are market reference data and belong to M1, §2.1). Where the brainstorm and the proposal conflict, the proposal wins. Where the brainstorm goes beyond the proposal, the addition is kept only if it earns its place, and §4.1 marks it as an extension rather than implying the proposal asked for it.

The one decision that is settled independently of both documents is the architectural style: **microservices**, by team decision (ARC §0 A1).

Requirement anatomy. Every requirement is a single, testable statement carrying four fields:

- **ID** — stable identifier, `FR-<SERVICE>-NNN` or `NFR-<CATEGORY>-NNN`. IDs are never reused; a withdrawn requirement is struck, not recycled.
- **Requirement** — one "the system shall …" statement, written to be verifiable (avoids vague verbs like "support" without an object, "handle", "manage").
- **Priority** — **Core** or **Extended** (defined below).
- **Trace** — the ARC section(s) and, where useful, the owning service database, that realize the requirement.

Priority tiers. Because the deployment posture is *cloud-ready by design but demonstrated locally* (ARC §0 A2), priority reflects what the team commits to demonstrating, not merely what is desirable.

- **Core** — the defensible baseline. The system does not fulfill its central premise (a simulation-driven risk-analytics platform with explainable, topology-aware analytics running end-to-end) without these. Every Core requirement is demonstrable on the local Docker Compose stack.
- **Extended** — genuinely in scope and designed for, built if time allows. These deepen a capability (more stochastic models, more stress scenarios, richer factor decomposition) or realize production-grade operational qualities (replica failover, PITR drills) that are specified and coded-for but not required for the core demonstration.

Verb convention. "Shall" denotes a binding requirement. "Should" is not used for requirements (it is reserved for non-binding guidance and does not appear in the numbered lists). Data-at-rest structures named in the Trace column (tables, service databases) are defined in ARC §8 and are not redefined here.

Reading the tables. Each service opens with a short prose statement of the requirement themes and the mechanics that make them non-trivial, followed by the numbered requirement table. The prose carries the *why*; the table carries the binding *what*.

1. Actors, roles & access model

QuantSim recognizes four roles. A request is **authenticated once at the API gateway**, which verifies the token and forwards the caller's identity; the **authorization decision is then made inside the service that owns the affected data**, on every route, failing closed if a scope cannot be resolved (ARC §9.1, §9.2). The role set itself is owned by the Platform/Identity service and pinned to a fixed enumeration at the database level (ARC §8.9). The roles form an escalating-capability model rather than strictly nested sets: each is defined by what work it performs.

Viewer consumes finished analytics — dashboards, reports, alerts — and can do nothing that changes system state. **Analyst** additionally runs analysis over existing artifacts: launches stress tests and TDA, computes strategy metrics, defines alert rules, and configures personal dashboards, but does not author trading strategies or ingest data. **Quant** additionally authors the inputs to the simulation core: strategies, model calibrations, simulation runs, and portfolios, and trains/registers ML models. **Admin** additionally administers the platform: user accounts, role assignment, instrument master data, and system configuration; Admin is an operator role, not a super-analyst, and holds the governance capabilities rather than a superset of every quant action.

The following matrix is the authoritative capability summary; individual FRs restate the specific role where it matters.

Capability area	Viewer	Analyst	Quant	Admin
View dashboards, replays, alerts	✓	✓	✓	✓
Run stress tests / TDA; compute metrics		✓	✓	✓
Define alert rules; customize own dashboard		✓	✓	✓
Author strategies, calibrations, simulations, portfolios			✓	✓
Train / register ML models			✓	✓
Ingest / configure market data sources			✓*	✓
Manage users, roles, instrument master, config				✓

*Quant may trigger ingestion for instruments already under management; adding a new tracked instrument or data source is Admin-only (see FR-MKT-001, FR-PLT-003).

Access to data rows (which portfolios/instruments a given user may see) is additionally scoped inside the owning service's data-access layer (ARC §9.2), independent of role: holding the Analyst role does not grant sight of every portfolio, only those the user is entitled to. Because entitlement is evaluated where the data lives, no service can be induced to release rows by a caller that has merely reached it.

2. Functional requirements

2.0 · M0 — Platform, Identity & Governance (cross-cutting)

This group covers the capabilities that do not belong to any one domain service but underpin all of them: account lifecycle, authentication, role-based authorization, and the audit trail. **M0 has no counterpart among the proposal's seven modules, and that is the reason it exists as a separate group.** The proposal describes a multi-user analytics platform whose outputs must remain "interpretable and auditable" but assigns identity, roles, and audit to no module. Those concerns are needed regardless, and they are genuinely cross-cutting: every one of M1–M7 authenticates callers, authorizes actions, and audits its own writes. There were two

ways to place them — distribute them across the seven modules, or isolate them — and isolating them is the efficient answer, because it keeps the proposal's seven modules intact and undistorted, gives token issuance and the role domain exactly one owner, and confines the credential and audit machinery to one bounded context instead of smearing it across seven (ARC §5.3). M0 is therefore an architectural addition, justified on that ground and not by any list of functional groups. The defining mechanic here is that governance is enforced at the lowest responsible layer — authentication once at the gateway, the authorization decision inside the service that owns the data, and the role domain and audit capture in each service's database — so no alternate code path can bypass it (ARC §9.1, §9.2, §8.7, §8.9). Platform/Identity is deliberately *not* a synchronous dependency of every request: it issues asymmetrically-signed tokens that other services verify locally, so an Identity outage stops logins and role changes without stopping the platform (ARC §9.1, §11.1).

ID	Requirement — *the system shall …*	Priority	Trace
FR-PLT-001	allow an Admin to create a user account with a unique username and email and a hashed password	Core	ARC §9.1, §8.2 `platform_db`
FR-PLT-002	allow an Admin to deactivate a user account without deleting it, and to later reactivate it, preserving the account's history, and propagate the deactivation so that already-issued tokens for that account cease to be honoured	Core	ARC §9.1
FR-PLT-003	allow an Admin to assign and change a user's role within the fixed role set (§1), publishing the change so services holding a cached entitlement observe it	Core	ARC §9.2, §7.4
FR-PLT-004	authenticate requests via an asymmetrically-signed JWT that any service can verify locally without calling the issuer, and associate every request with the acting user across every service the request reaches	Core	ARC §9.1
FR-PLT-005	authorize every state-changing and data-reading action against the acting user's role and row-scope, inside the service that owns the affected data, before the action proceeds	Core	ARC §9.2
FR-PLT-006	automatically record an audit entry capturing before- and after-state for every insert, update, and delete on business data, excluding the designated append-only tables	Core	ARC §8.7
FR-PLT-007	retain audit entries for deleted rows such that what was deleted remains queryable after the row is gone	Core	ARC §8.7
FR-PLT-008	reconstruct the complete, ordered change history of any single business record from the audit trail	Core	ARC §8.7

2.1 · M1 — Market Data & Signal Ingestion

M1 acquires and conditions every input the rest of the platform consumes, and it owns the abstraction on which the whole premise rests: a **market environment** that a portfolio can be run against, whether that environment is a replayed historical period or a synthetic simulation (ARC §2 P1, §5 M1). Three mechanics

make this group non-trivial. Ingestion must be **idempotent** — re-running a window must not double-count. Ingested data must be **cleaned and normalized before publication** (FR-MKT-011), which the proposal names as a module responsibility and which distribution turns into a stronger obligation: once a bar is published as an event, seven services may have consumed it, so a bad tick corrected late is a correction that has to propagate rather than a row that can be quietly fixed. And derived quantities (returns, volatility, correlations) are **computed once, stored, and published** so five downstream services do not each recompute them differently (ARC §8.4, §7.4). Under microservices that third mechanic carries more weight than it did in version 0.1: because no other service can join against `market_db`, Market Data is the single authority for what a price *was*, and every consumer either calls its versioned API or maintains a read model fed by its events (ARC §2 P8, §8.12).

One consequence of the compute-once rule is worth stating here because it settles an ownership question that would otherwise be answered twice. The **rolling correlation matrices the TDA pipeline consumes are Market Data's output, not Risk's** (FR-MKT-004, FR-RSK-011): correlation is a derived market-data quantity, ML's regime classifier wants it too, and a correlation matrix over a closed historical window is immutable and therefore cacheable indefinitely (ARC §5.4, §9.4). Risk requests the series for its chosen windows in one batched call and fingerprints what it received into the analysis window, so a persistence diagram stays reproducible against an exact input (ARC §8.10). Had Risk computed its own correlations from a local returns projection it would have been the second implementation of the same statistic in the platform — the precise outcome FR-MKT-003 exists to prevent.

ID	Requirement — *the system shall …*	Priority	Trace
FR-MKT-001	allow an Admin to maintain the master list of tradable instruments (add, annotate, and non-destructively retire)	Core	ARC §8.2 `market_db`
FR-MKT-002	ingest OHLCV price bars for every tracked instrument on a schedule, recording per-run status, row count, and errors, without duplicating rows on re-ingestion of the same window	Core	ARC §5 M1, §8.4
FR-MKT-003	precompute and store per-instrument returns and volatility from ingested bars, and publish them, so downstream services consume stored or received values rather than recomputing them	Core	ARC §8.4, §7.4
FR-MKT-004	allow definition of a named basket of instruments and generate a pairwise correlation matrix for that basket over a specified time window, and return the **rolling series** of such matrices for a given window length and step in a single request, so a consumer analysing many consecutive windows does not issue one call per window	Core	ARC §5.4, §8.10
FR-MKT-005	maintain a basket as a named, versioned entity with a full history of its membership changes	Extended	ARC §8.2
FR-MKT-006	ingest financial news articles and automatically link each article to the instrument(s) it references	Core	ARC §5 M1
FR-MKT-007	allow an authorized user to create a pseudo-real-time replay	Core	ARC §5 M1, §7.3

ID	Requirement — *the system shall …*	Priority	Trace
	session over a historical period and to play, pause, and set the playback speed of that session		
FR-MKT-008	reject a replay session whose end time precedes its start time	Core	ARC §8.9
FR-MKT-009	expose replayed-historical and synthetic-simulated markets through one common market-environment interface, so a portfolio can be bound to either without change	Core	ARC §2 P1, §5 M1
FR-MKT-010	record corporate actions (splits, mergers, dividends) for tracked instruments and provide split/dividend-adjusted price series for analysis	Extended	ARC §15 (open Q)
FR-MKT-011	clean and normalize every ingested series before it is published — detecting and recording missing bars, duplicate bars, non-positive or out-of-range prices, and zero-volume anomalies; aligning all timestamps to UTC against the instrument's trading calendar; and normalizing ingested news text (encoding, deduplication, boilerplate removal) — such that each ingestion run reports what it corrected, what it rejected, and what it passed through untouched	Core	ARC §5.4, §8.4

2.2 · M2 — Stochastic Market Dynamics Simulator

M2 manufactures the synthetic side of the market-environment abstraction. It fits established stochastic models to real data (so synthetic markets are grounded, not arbitrary) and generates Monte Carlo ensembles from them. The group is split so the four model families can be prioritized independently — the classic trio (GBM, mean-reversion, jump-diffusion) is Core because it covers drift, reversion, and discontinuous shocks; regime-switching is Extended because it is the most complex to calibrate. A hard correctness edge is enforced in storage: a run cannot be simulated against an instrument its calibration was not fitted to (ARC §8.9).

ID	Requirement — *the system shall …*	Priority	Trace
FR-SIM-001	calibrate a supported stochastic model to a chosen instrument over a chosen historical window and persist the fitted parameters keyed by calibration type	Core	ARC §5 M2, §8.2 `simulation_db`
FR-SIM-002	support Geometric Brownian Motion (GBM)	Core	ARC §5 M2
FR-SIM-003	support the Ornstein–Uhlenbeck mean-reversion process	Core	ARC §5 M2
FR-SIM-004	support a jump-diffusion model	Core	ARC §5 M2
FR-SIM-005	support regime-switching, allowing definition of distinct regimes (each with its own drift and volatility) and the transition probabilities between them	Extended	ARC §5 M2

ID	Requirement — *the system shall …*	Priority	Trace
FR-SIM-006	generate a Monte Carlo ensemble of N paths over T time steps from a chosen calibration, persisting the simulated value at every step of every path	Core	ARC §5 M2, §6, §8.4
FR-SIM-007	reject a simulation run whose target instrument differs from the instrument of its source calibration	Core	ARC §8.9
FR-SIM-008	precompute and store summary statistics (mean terminal price, VaR, CVaR) for every simulation run	Core	ARC §5 M2
FR-SIM-009	stream progress of a running simulation to the requesting client	Extended	ARC §6, §7.3

2.3 · M3 — Event-Driven Trading Engine

M3 executes strategies against a bound market environment in strict chronological order and records the result in a form that can be replayed exactly. Its defining mechanic is **event sourcing**: the append-only, per-portfolio, strictly-ordered event log is the source of truth, and the portfolio's value, cash, and holdings series are append-only *projections* derived from it (ARC §8.5). This is what turns "reconstruct a portfolio's history exactly" from a feature into a property of the data model. The log is **private to the Trading service**: no other service reads it, and consumers learn about executions through published `fill.*` and `portfolio.*` events or a point-in-time state query, which is what keeps strict ordering enforceable in one place (ARC §5.6, §8.5, §2 P8). Realistic execution — transaction costs and slippage — is required because a backtest that ignores them overstates returns.

ID	Requirement — *the system shall …*	Priority	Trace
FR-TRD-001	allow a Quant to define a trading strategy that references its logic and its configuration parameters	Core	ARC §5 M3, §8.2 `trading_db`
FR-TRD-002	allow a Quant to create a portfolio with a starting capital, a base currency, an associated strategy, and a link to a simulation run or a replay session	Core	ARC §5 M3, §8.5
FR-TRD-003	apply a configurable transaction-cost model (fixed, per-share, and percentage) to every order execution	Core	ARC §5 M3
FR-TRD-004	additionally support a tiered transaction-cost model	Extended	ARC §5 M3
FR-TRD-005	apply a configurable slippage model to order execution	Core	ARC §5 M3
FR-TRD-006	maintain an immutable, strictly time-ordered log of every trading event (order placed, filled, cancelled, market tick) sufficient to replay a portfolio's history exactly	Core	ARC §8.5
FR-TRD-007	maintain a time series of each portfolio's total value, cash	Core	ARC §8.5

ID	Requirement — *the system shall …*	Priority	Trace
	balance, and profit/loss as an append-only projection of the event log		
FR-TRD-008	maintain a complete, non-destructive history of portfolio holdings over time	Core	ARC §8.5
FR-TRD-009	record the actual fill price, filled quantity, and realized cost for every executed order	Core	ARC §8.5
FR-TRD-010	support placing market and limit orders within a portfolio	Core	ARC §5 M3
FR-TRD-011	support placing stop and stop-limit orders within a portfolio	Extended	ARC §5 M3

2.4 · M4 — Machine Learning & Financial Intelligence

M4 turns market and text data into predictions. Two requirements here carry correctness weight far beyond the rest and are therefore Core infrastructure regardless of which predictive tasks ship: a **model registry** that makes every prediction attributable to an exact, reproducible model version, and a **point-in-time feature store** that makes look-ahead bias structurally impossible rather than a discipline the modeller must remember (ARC §8.6). Under microservices that second guarantee gets stronger rather than weaker: because ML cannot join against another service's tables, it materializes features from the events it receives and stamps the moment each fact *became known*, which is precisely the timestamp look-ahead prevention needs (ARC §5.8, §8.6). Among the predictive tasks themselves, the deterministic and text-driven ones (strategy metrics, sentiment, volatility) are Core; the composite and classification tasks that need labeled outcomes are Extended.

ID	Requirement — *the system shall …*	Priority	Trace
FR-ML-001	maintain a registry of ML models recording each model's task type, its trained versions, their hyperparameters, and the reference path to each version's stored artifact	Core	ARC §8.6
FR-ML-002	generate point-in-time feature snapshots such that a snapshot as-of time *t* exposes only data whose knowledge-time is at or before *t*	Core	ARC §8.6
FR-ML-003	record training-run metadata — loss curve, final metrics, and status — for each model version	Core	ARC §8.6
FR-ML-004	produce a sentiment score for each ingested news article, attributable to a specific model version	Core	ARC §5 M4
FR-ML-005	produce a forward-looking volatility forecast per instrument	Core	ARC §5 M4
FR-ML-006	compute strategy performance metrics — Sharpe ratio, win rate, alpha, and beta — for a portfolio and its strategy	Core	ARC §5 M4

ID	Requirement — *the system shall …*	Priority	Trace
FR-ML-007	extract structured financial events (for example layoffs, earnings surprises) from news-article text	Extended	ARC §5 M4
FR-ML-008	produce a current market-regime classification per instrument	Extended	ARC §5 M4
FR-ML-009	produce a drawdown-risk prediction for a portfolio, including a probability estimate	Extended	ARC §5 M4
FR-ML-010	produce a single composite portfolio-health score together with a supporting breakdown	Extended	ARC §5 M4

2.5 · M5 — Stress Testing & Market Structure Analysis (TDA)

M5 has two halves behind one service boundary and one database (`risk_db`). **Stress testing** injects designed catastrophes and measures the damage and its attribution. **Topological Data Analysis** studies the *shape* of the market's correlation structure to detect regime transitions that conventional indicators miss — the platform's principal novelty. The stress-scenario catalog is registry-driven so a new scenario is a configuration row, not a migration (ARC §8.2 P5). The TDA pipeline is specified stage-by-stage because each stage's artifact is persisted (ARC §8.10) — that is what lets the dashboard explain *why* a risk score moved and lets consecutive windows be compared to flag a transition. Because TDA is the differentiator, the full pipeline is Core; breadth of stress scenarios and of breakdown dimensions is where the Core/Extended line falls. One distinction is drawn deliberately: the proposal lists **risk-concentration metrics** as an output metric alongside drawdown, exposure breakdown, and recovery trajectory, and concentration is not the same measurement as loss attribution. Attribution (FR-RSK-006) answers "what hurt us in this scenario"; concentration (FR-RSK-018) answers "how exposed are we to any one thing going wrong at all", and a portfolio can score badly on the second while the first looks unremarkable. Both are required.

ID	Requirement — *the system shall …*	Priority	Trace
FR-RSK-001	provide a registry-driven catalog of stress scenarios to which a new scenario can be added without a schema change	Core	ARC §8.2, §5 M5
FR-RSK-002	support flash-crash and volatility-spike stress scenarios	Core	ARC §5 M5
FR-RSK-003	support liquidity-shock, sector-wide-collapse, and correlated-asset-failure stress scenarios	Extended	ARC §5 M5
FR-RSK-004	execute a configured stress scenario against a specified portfolio, optionally on top of a synthetic simulated market	Core	ARC §5 M5, §6
FR-RSK-005	report maximum drawdown, VaR, and recovery days as results of a stress run	Core	ARC §5 M5
FR-RSK-006	report each instrument's contribution to a stress run's losses	Core	ARC §5 M5

ID	Requirement — *the system shall …*	Priority	Trace
FR-RSK-007	report stress results broken down asset-by-asset	Core	ARC §5 M5
FR-RSK-008	additionally break stress results down by sector and by geography	Extended	ARC §5 M5
FR-RSK-009	report the portfolio's value at each step of its post-shock recovery trajectory	Core	ARC §5 M5
FR-RSK-010	allow an authorized user to define the time window (bucket) for a TDA analysis	Core	ARC §8.10
FR-RSK-011	obtain the rolling correlation matrices for an analysis window from the owning Market Data service (FR-MKT-004), convert each into a distance matrix, and construct a Vietoris–Rips simplicial complex from it, recording the fingerprint of the source matrix it used	Core	ARC §8.10
FR-RSK-012	compute persistent homology from the complex, identifying topological features (connected components, loops) and how long each persists across increasing thresholds	Core	ARC §8.10
FR-RSK-013	summarize a persistence diagram into Betti numbers at each threshold	Core	ARC §8.10
FR-RSK-014	transform a persistence diagram into a persistence landscape enabling comparison across analysis windows	Core	ARC §8.10
FR-RSK-015	compute a single composite topological-risk score per analysis window from that window's persistence diagram, Betti numbers, and landscape	Core	ARC §8.10
FR-RSK-016	detect and flag meaningful topological structural changes between consecutive analysis windows	Core	ARC §5 M5, §8.10
FR-RSK-017	surface the topological-risk score, its trend, and any flagged transitions to the dashboard and make them available to alert rules	Core	ARC §5 M7, §9.6
FR-RSK-018	compute risk-concentration metrics for a portfolio — the largest single-instrument and largest sector share of exposure, and a concentration index over the exposure distribution — and report them both for the portfolio's current state and as measured under a stress run	Core	ARC §5 M5

2.6 · M6 — Explainable Financial Reasoning (XAI)

M6 ensures no analytical output is a black box: it attributes predictions to features, losses to instruments, and risk to factors, and it narrates the result in plain language. SHAP attribution and natural-language

narration are Core because interpretability is one of the platform's three headline bets (ARC §16); factor-level risk decomposition and news-event-to-impact linkage are Extended because they depend on additional reference data (factor exposures, reliable event timestamps) that may not be available for every instrument.

ID	Requirement — *the system shall …*	Priority	Trace
FR-XAI-001	produce SHAP-based feature attributions for any model prediction it generates	Core	ARC §5 M6, §8.2 `xai_db`
FR-XAI-002	identify which instruments contributed most to a portfolio's profit or loss	Core	ARC §5 M6
FR-XAI-003	decompose a portfolio's risk by contributing factors such as sector and interest rates	Extended	ARC §5 M6
FR-XAI-004	link a specific news event to its measured price or volatility impact where such a relationship can be established	Extended	ARC §5 M6
FR-XAI-005	generate a plain-English natural-language explanation of a prediction, alert, or stress result via an LLM API, falling back to a cached or templated explanation if the provider is unavailable	Core	ARC §5 M6, §9.6

2.7 · M7 — Analytics Dashboard & Alerting

M7 is the single interface over everything above, and the alerting engine that watches the domain-event stream on the user's behalf. It is read-heavy and latency-bound, so it is built as a **read-model service**: it maintains its own event-fed projections of what the other services have published and answers a widget from its own database rather than fanning out live calls to five services, which is what makes the 2-second target achievable at all under decomposition (ARC §2 P6, §5.10, §8.8, §8.12). The cost of that choice is stated rather than hidden — its data is eventually consistent and its age is exposed (NFR-INT-007). The proposal names nine views for this module and all nine are carried here, with the Core/Extended line drawn by whether a view needs anything the platform is not already required to compute and store. Risk heatmaps and volatility analysis are **Core** on that test: their inputs — the correlation matrices of FR-MKT-004 and the volatility of FR-MKT-003 and FR-ML-005 — are Core and already persisted, so the view is a rendering rather than new work, and a risk platform that cannot draw a risk heatmap is hard to defend. Strategy comparison is Extended because it presupposes several completed portfolios to compare. Personalization (saved layouts, per-widget configuration) is Extended because nothing depends on it.

Alerting is the one part of M7 that goes **beyond** the proposal, which describes this module as a dashboard and stops there. It is kept because a risk platform whose user must be watching the screen to learn that a drawdown threshold was breached has not finished the job, and because the events it needs are already on the backbone for other reasons — the marginal cost is a rule evaluator, not a new pipeline. Alert delivery is push, never poll (ARC §9.6), and every trigger is recorded for accountability. One property of alerting is stated as a requirement rather than left to design, because it is what users actually judge the feature by: an alert must fire **once per condition, not once per event** (FR-DSH-009). Under at-least-once delivery the same event can be seen twice, a restarted evaluator replays from its last offset, and a metric sitting on its threshold will

cross it repeatedly — all three produce a flood of identical notifications unless the rule carries state and a cooldown, and a flood is indistinguishable from noise, which is how alerting features get switched off.

ID	Requirement — *the system shall …*	Priority	Trace
FR-DSH-001	present, in the dashboard, portfolio overview, trade-replay visualization, drawdown, stress-test results, market-regime, and topological-risk views	Core	ARC §5 M7
FR-DSH-002	additionally present risk heatmaps and volatility analysis	Core	ARC §5 M7
FR-DSH-003	allow a user to save a personal dashboard layout	Extended	ARC §5 M7
FR-DSH-004	allow a user to configure individual widgets within a layout	Extended	ARC §5 M7
FR-DSH-005	allow an authorized user to define alert rules on portfolio and/or risk conditions (for example a drawdown threshold or a topological-risk threshold)	Core	ARC §9.6
FR-DSH-006	record every alert-trigger event, including whether and when the user acknowledged it	Core	ARC §9.6
FR-DSH-007	deliver alert notifications to connected clients by server push, without the user manually refreshing	Core	ARC §9.6
FR-DSH-008	present strategy-comparison tools that place two or more strategies' performance metrics and value series side by side	Extended	ARC §5 M7
FR-DSH-009	notify a user **once** per alert condition rather than once per qualifying event — holding a rule in a *firing* state until the condition is observed clear again, re-notifying only after a configured cooldown, and discarding duplicate evaluations of the same rule against the same subject and window	Core	ARC §5.10, §9.6

3. Non-functional requirements

Every NFR below is stated to be **measurable or objectively checkable** — a target, a threshold, or a binary property — so it can serve as an acceptance gate rather than an aspiration. Priority follows the deployment posture (ARC §0 A2): qualities demonstrable on the local stack are Core; production-grade operational qualities (replica failover, point-in-time recovery, HA broker) are specified and coded-for but tiered Extended, because they are not required for the core end-to-end demonstration. Each NFR's Trace cites the ARC mechanism that delivers it — the reverse direction of ARC §12's traceability matrix.

Because the architecture is **microservices** (ARC §0 A1), a whole class of non-functional requirement exists here that a single-process system would not need: the platform must stay correct when one of eight services is unavailable, must converge when data is replicated across service boundaries, must survive duplicate event delivery, and must remain traceable across a request that touches four processes. Those requirements are

not decorative — they are the accepted cost of A1 written down as acceptance gates, and they are concentrated in §3.3 (REL), §3.4 (INT), §3.8 (OBS), and §3.10 (MNT).

3.1 · Performance (PERF)

ID	Requirement — *the system shall …*	Priority	Trace
NFR-PERF-001	return dashboard widget queries within 2 seconds at p95 under normal load, reading precomputed data from the Dashboard service's own store without a synchronous call to another service	Core	ARC §8.4, §8.12, §9.4
NFR-PERF-002	complete standard CRUD API operations within 800 ms at p95 under normal load, measured end-to-end through the API gateway	Core	ARC §8.11, §5.2
NFR-PERF-003	when audit triggers are enabled on a table, increase that table's write latency by no more than 20% versus the same table unaudited	Core	ARC §8.7
NFR-PERF-004	acknowledge a long-running job submission (enqueue) within 800 ms, with the computation itself proceeding asynchronously	Core	ARC §6
NFR-PERF-005	bound every synchronous inter-service call with an explicit timeout, such that an unresponsive dependency degrades the caller's latency by no more than that timeout rather than blocking it indefinitely	Core	ARC §7.5

3.2 · Scalability (SCAL)

ID	Requirement — *the system shall …*	Priority	Trace
NFR-SCAL-001	allow simulation/compute throughput to scale by adding worker instances without degrading any service's API p95 latency	Core	ARC §6, §10.3
NFR-SCAL-002	scale dashboard read capacity independently of the write path, through event-fed local read models and, in production, a read replica	Extended	ARC §8.8, §8.12
NFR-SCAL-003	allow a new ML model type, ML task type, or stress scenario to be added with no schema migration	Core	ARC §8.6, §8.2
NFR-SCAL-004	parallelize Monte Carlo path generation across available cores and worker instances	Core	ARC §6
NFR-SCAL-005	permit each service to be scaled to a different replica count independently, with no change to and no redeployment of any	Core	ARC §10.2, §10.3

ID	Requirement — *the system shall …*	Priority	Trace
	other service		

3.3 · Reliability & Availability (REL)

ID	Requirement — *the system shall …*	Priority	Trace
NFR-REL-001	in production, maintain a database replica capable of serving reads and being promoted on primary failure	Extended	ARC §11.2
NFR-REL-002	support point-in-time recovery with a recovery-point objective under 5 minutes, per service database	Extended	ARC §11.2
NFR-REL-003	verify backup restorability through scheduled restore drills into a non-production environment, treating no backup as valid until a restore succeeds	Extended	ARC §11.2
NFR-REL-004	configure the event backbone so that a single broker failure causes no event loss	Extended	ARC §7.4, §11.2
NFR-REL-005	make compute jobs idempotent and reclaimable, so a worker crash mid-job neither corrupts results nor loses the job	Core	ARC §6
NFR-REL-006	for every service-to-service dependency, define and implement a declared degraded behaviour — stale-with-age, queued, or explicitly-unavailable — such that an unavailable dependency never causes the caller to hang, to cascade, or to return a value it cannot substantiate	Core	ARC §7.5, §11.1
NFR-REL-007	commit a state change and its resulting published event atomically, such that a broker outage delays event delivery but never loses an event and never blocks the local write	Core	ARC §7.7, §11.1
NFR-REL-008	process every event at-least-once with idempotent effect, such that redelivery of an already-handled event produces no additional state change	Core	ARC §7.7, §11.3
NFR-REL-009	for every workflow spanning two or more service databases, implement a compensating action for each step and reach either a completed or an explicitly-recorded failed state, never a silently partial one	Core	ARC §7.6, §11.3
NFR-REL-010	restore any single service independently of the other seven, reconciling to a consistent state by replaying the retained event stream from its restored offset	Extended	ARC §11.2

3.4 · Data Integrity & Consistency (INT)

ID	Requirement — *the system shall …*	Priority	Trace
NFR-INT-001	store all monetary and price values in fixed-precision decimal types, never binary floating point, in every service database	Core	ARC §8.9
NFR-INT-002	constrain the user-role attribute to the set {admin, quant, analyst, viewer} at the database level, not only in application code	Core	ARC §8.9
NFR-INT-003	reject, at the database level, a replay session whose end time precedes its start time	Core	ARC §8.9
NFR-INT-004	reject, at the data layer, a simulation run whose instrument does not match its calibration's instrument	Core	ARC §8.9
NFR-INT-005	enforce append-only semantics on event-log, projection, and audit tables by withholding UPDATE and DELETE privileges	Core	ARC §8.5, §8.9
NFR-INT-006	uphold every cross-service reference — which cannot be a foreign key — by validating it against the owning service on write, reacting to the owner's invalidation events, and reconciling on a scheduled sweep that reports any orphan it finds rather than silently repairing it	Core	ARC §8.9
NFR-INT-007	converge every event-fed read model to its source within a stated bound under normal load, and expose the age of the data it served, so a consumer can distinguish stale from current instead of assuming currency	Core	ARC §8.12, §9.5

3.5 · Security (SEC)

ID	Requirement — *the system shall …*	Priority	Trace
NFR-SEC-001	authenticate every external request once at the API gateway and enforce every authorization decision inside the service that owns the affected data, failing closed when a scope cannot be resolved	Core	ARC §9.1, §9.2
NFR-SEC-002	encrypt all traffic in transit with TLS — client-to-gateway, gateway-to-service, service-to-service, and service-to-datastore	Core	ARC §9.3
NFR-SEC-003	store database credentials and third-party API keys in a dedicated secret manager, with no secret present in source or container images	Core	ARC §9.3
NFR-SEC-004	store user passwords only as salted one-way hashes, never as plaintext or reversible ciphertext	Core	ARC §8.2

ID	Requirement — *the system shall …*	Priority	Trace
NFR-SEC-005	keep the audit log queryable after a row is deleted, such that it reports what was deleted	Core	ARC §8.7
NFR-SEC-006	authenticate the calling service on every internal call, carrying the originating user's identity separately from the service's own identity so authorization decisions are made about the user, not the caller	Core	ARC §7.8, §9.1
NFR-SEC-007	issue each service its own database credentials limited to its own database, such that compromise of one service grants no access to another service's data	Core	ARC §8.2, §9.3

3.6 · Recovery & Continuity (REC)

ID	Requirement — *the system shall …*	Priority	Trace
NFR-REC-001	rebuild each service's complete database schema from that service's own migration history alone, with no manual DDL	Core	ARC §8.11
NFR-REC-002	rebuild the entire environment — all services, all databases, and the event backbone — from infrastructure-as-code plus migration history, independent of any single engineer's manual knowledge	Extended	ARC §10.2, §11.4
NFR-REC-003	rebuild any event-fed read model from the retained event stream without requiring a backup of that read model	Core	ARC §8.12, §11.2

3.7 · Auditability (AUD)

ID	Requirement — *the system shall …*	Priority	Trace
NFR-AUD-001	record automatically, with before- and after-state, every insert, update, and delete on audited business data, within the service database that owns it (coverage guarantee for FR-PLT-006)	Core	ARC §8.7
NFR-AUD-002	make the complete change history of any single record reconstructable from the owning service's audit trail	Core	ARC §8.7
NFR-AUD-003	publish an audit-index event for each recorded change so that an investigation can locate which service holds a given change, without asserting a single globally-ordered audit log across services	Core	ARC §8.7

3.8 · Observability (OBS)

ID	Requirement — *the system shall …*	Priority	Trace
NFR-OBS-001	propagate one correlation id and trace context across every hop of a request — gateway, service-to-service call, job queue, and event consumption — so a causal chain spanning multiple services is reconstructable end-to-end	Core	ARC §9.5
NFR-OBS-002	emit structured logs and operational metrics from every service, including per-dependency call latency and error rate, circuit-breaker state, retry and compensation counts, queue depth, job duration, and replication lag	Core	ARC §9.5
NFR-OBS-003	expose event-consumer lag per consumer group and outbox drain backlog as first-class metrics, so the staleness of derived data is measured rather than assumed	Core	ARC §8.12, §9.5

3.9 · Accessibility & Usability (ACC)

ID	Requirement — *the system shall …*	Priority	Trace
NFR-ACC-001	present a plain-language explanation alongside raw metrics in dashboard visualizations, so a non-technical user can interpret them — and still render the metrics when the explanation service is unavailable	Core	ARC §5.9, §9.6, §11.1
NFR-ACC-002	deliver alert notifications by push rather than requiring the user to refresh, falling back to polling if the push connection drops	Core	ARC §9.6, §11.1

3.10 · Maintainability & Portability (MNT)

ID	Requirement — *the system shall …*	Priority	Trace
NFR-MNT-001	inject all configuration from the environment or secret manager (twelve-factor), running the same build artifact in every environment	Core	ARC §9.7
NFR-MNT-002	give each service exclusive ownership of its own database, with no shared tables, no cross-service foreign keys, and no cross-service joins — a service's data being reachable only through its published API or its emitted events	Core	ARC §2 P8, §8.2
NFR-MNT-003	~~access the event backbone through an interface that permits substituting the broker (Kafka in production, Redis Streams locally) without domain-code change~~ — **withdrawn** at v0.2: the backbone now carries cross-service consistency, so	—	—

ID	Requirement — *the system shall …*	Priority	Trace
	development and production must exercise identical broker semantics (ARC §0, AD-15)		
NFR-MNT-004	build, test, and deploy each service through its own pipeline, such that releasing one service requires no code change to, and no redeployment of, any other service	Core	ARC §10.2, §8.11
NFR-MNT-005	evolve every published API and event schema additively within a major version, and verify compatibility by running each consumer's contract tests in the provider's own pipeline before release	Core	ARC §7.1, §7.4
NFR-MNT-006	implement resilience, tracing, outbox publication, and migration scaffolding once in a shared service template inherited by every service, rather than separately per service	Core	ARC §5.1

4. Coverage & traceability

4.1 Proposal-to-requirement coverage

This matrix is the binding coverage claim of this document: **every capability named in the approved seven-module proposal is carried by at least one requirement here.** Nothing from the proposal is silently dropped, renamed away, or left to be inferred, and where the proposal was terse a requirement makes it testable. It is organized by the proposal's own module list, in the proposal's own order, so it can be checked against the source document line by line.

Proposal module	Proposal capability	Covering requirement(s)
M1	Historical OHLCV / index / equity ingestion	FR-MKT-002
M1	Data cleaning & normalization	FR-MKT-011
M1	Pseudo real-time market replay	FR-MKT-007
M1	Return computation	FR-MKT-003
M1	Volatility extraction	FR-MKT-003
M1	Correlation matrix generation	FR-MKT-004
M1	Financial news ingestion & preprocessing	FR-MKT-006, FR-MKT-011
M2	Geometric Brownian Motion	FR-SIM-002
M2	Ornstein–Uhlenbeck mean reversion	FR-SIM-003

Proposal module	Proposal capability	Covering requirement(s)
M2	Jump-diffusion	FR-SIM-004
M2	Regime-dependent parameter switching	FR-SIM-005
M2	Historical parameter calibration	FR-SIM-001
M3	Market / limit order execution	FR-TRD-010
M3	Stop order execution	FR-TRD-011
M3	Transaction-cost modelling	FR-TRD-003, FR-TRD-004
M3	Slippage simulation	FR-TRD-005
M3	Portfolio state updates	FR-TRD-007, FR-TRD-008
M3	Trade logging & replay	FR-TRD-006
M4	News sentiment analysis	FR-ML-004
M4	Event extraction from text	FR-ML-007
M4	Market-regime classification	FR-ML-008
M4	Volatility forecasting	FR-ML-005
M4	Drawdown-risk prediction	FR-ML-009
M4	Portfolio health scoring	FR-ML-010
M4	Strategy performance / evaluation	FR-ML-006, FR-XAI-002
M4	Potential methods: XGBoost, Random Forest, LSTM, Transformer financial NLP	*Not stated as requirements* — method choice is a design matter, realized in the stack (ARC §13) and constrained by FR-ML-001/003 to be recorded per model version
M5	Flash-crash / volatility-spike simulation	FR-RSK-002
M5	Liquidity-shock / sector-collapse / correlated-failure	FR-RSK-003
M5	Drawdown analysis	FR-RSK-005
M5	Exposure breakdown	FR-RSK-007, FR-RSK-008
M5	Recovery-trajectory analysis	FR-RSK-009

Proposal module	Proposal capability	Covering requirement(s)
M5	Risk-concentration metrics	FR-RSK-018
M5	Rolling correlation → distance → Vietoris–Rips	FR-MKT-004, FR-RSK-011
M5	Persistent homology	FR-RSK-012
M5	Betti-number analysis	FR-RSK-013
M5	Persistence landscapes	FR-RSK-014
M5	Market-regime transition detection	FR-RSK-016
M5	Topological risk score	FR-RSK-015, FR-RSK-017
M6	SHAP feature attribution	FR-XAI-001
M6	Asset-level contribution analysis	FR-XAI-002
M6	Risk-source decomposition	FR-XAI-003
M6	Event-to-impact traceability	FR-XAI-004
M6	Natural-language explanation	FR-XAI-005
M7	Portfolio overview / trade replay / drawdown / stress / regime / topo views	FR-DSH-001
M7	Risk heatmaps / volatility analysis	FR-DSH-002
M7	Strategy comparison tools	FR-DSH-008

Deliberate additions beyond the proposal. The following requirements have no counterpart in the seven-module proposal. Each originates in the team's internal brainstorm or in an architectural necessity, and each is kept because it earns its place — listed here so no reader mistakes an addition for something the proposal asked for, and so the team can strike any of them without touching proposal coverage.

Addition	Requirements	Why it is kept
Identity, roles, RBAC, account lifecycle	FR-PLT-001…005	The proposal describes a multi-user platform and assigns user management to no module; four distinct roles make the access model statable and testable (§1, §2.0)
Audit trail with before/after state	FR-PLT-006…008	The proposal requires outputs to remain "auditable"; a trigger-level trail is what makes that a property rather than a claim
Instrument master & versioned baskets	FR-MKT-001, FR-MKT-005	Correlation matrices and portfolios both need a defined universe; the proposal assumes one without specifying it

Addition	Requirements	Why it is kept
Corporate actions	FR-MKT-010	Raw prices are wrong across splits and dividends, which would silently corrupt returns, volatility, and every correlation built on them
Common market-environment interface	FR-MKT-009	Makes the proposal's central claim — run a portfolio against replayed *or* synthetic markets — a single binding contract rather than two parallel code paths
Model registry & training metadata	FR-ML-001, FR-ML-003	Attribution of a prediction to an exact model version; without it no ML output is reproducible
Point-in-time feature store	FR-ML-002	Look-ahead bias is the failure mode that most quietly invalidates financial ML; structural prevention beats discipline
Stress-scenario registry	FR-RSK-001	Lets a new scenario be configuration rather than a schema migration
Instrument-level loss attribution in stress runs	FR-RSK-006	The proposal asks the *explainability* module which assets contributed to losses (FR-XAI-002); computing it where the stress run happens is cheaper than reconstructing it afterwards
Alert rules, trigger log, push delivery	FR-DSH-005…007	A risk platform whose user must be watching the screen to learn of a breach has not finished the job (§2.7)
Alert deduplication and firing state	FR-DSH-009	At-least-once delivery and threshold oscillation both produce repeat notifications; an alert channel that cries wolf gets muted, which costs the whole feature
Dashboard personalization	FR-DSH-003, FR-DSH-004	Low-cost usability; the first requirements to cut if time binds
Correctness guards	FR-MKT-008, FR-SIM-007	Cheap database-level checks against two errors that are otherwise silent (ARC §8.9)

4.2 Owning-service and data-ownership register

Every FR anchors to exactly one **owning service**, which under microservices means one owning deployable unit and one owning database (ARC §2 P8, §8.2) — so this table doubles as the data-ownership register. Each FR's Trace column (in §2) points at the ARC section that builds it. The correspondence between the proposal's modules and the services is 1:1 by construction, which is the property that lets one vocabulary serve the proposal, this document, and the architecture (ARC §3).

Requirement group	Proposal module	Owning service	Owning database (ARC §8.2)
M0 (FR-PLT-*)	*none — architectural addition*	Platform / Identity	`platform_db`

Requirement group	Proposal module	Owning service	Owning database (ARC §8.2)
M1 (FR-MKT-*)	Module 1 — Market Data & Signal Ingestion Engine	Market Data	`market_db`
M2 (FR-SIM-*)	Module 2 — Stochastic Market Dynamics Simulator	Simulation	`simulation_db`
M3 (FR-TRD-*)	Module 3 — Event-Driven Trading Engine	Trading	`trading_db`
M4 (FR-ML-*)	Module 4 — Machine Learning & Financial Intelligence Layer	ML	`ml_db`
M5 (FR-RSK-*)	Module 5 — Stress Testing & Market Structure Analysis Engine	Risk & TDA	`risk_db`
M6 (FR-XAI-*)	Module 6 — Explainable Financial Reasoning Engine	Explainability	`xai_db`
M7 (FR-DSH-*)	Module 7 — Analytics Dashboard	Dashboard & Alerting	`dashboard_db`

Two placements differ from the team brainstorm's grouping and the difference is deliberate. The brainstorm filed the **instrument master and basket definitions under "Authentication"**; they are market reference data, every correlation and portfolio depends on them, and nothing about them is an identity concern — they belong to M1, which is also where the proposal puts them. And the brainstorm had no separate identity component at all, placing authentication *and* every authorization decision in the API gateway; M0 exists as a service instead, and authorization moved into the owning services (§2.0, ARC §9.2).

Two consequences of this table are binding rather than descriptive. First, where a requirement in one row needs data owned by another row, it is satisfied through that service's published API or its emitted events — never a join — and any locally held copy is a read model, not a second source of truth (NFR-MNT-002, NFR-INT-007). Second, the audit trail is **per service**: each row's database holds the change history for its own data, and cross-service investigation goes through the audit-index events rather than a single global log (NFR-AUD-001, NFR-AUD-003, ARC §8.7).

5. Assumptions, exclusions & open questions

These are the boundaries within which the requirements above hold. Each needs a team or advisor decision; none blocks starting on the Core set. They inherit and extend the open questions from ARC §15.

Data sourcing is pseudo-real-time, not live. "Replay" and scheduled ingestion assume historical data from free or academic sources (for example end-of-day or delayed bars); no paid live-tick market feed is in scope. Confirming this matters because a live feed changes FR-MKT-002/007 and the cost model materially.

Corporate actions are Extended, and the adjustment policy is open. FR-MKT-010 commits to recording splits, mergers, and dividends and providing adjusted series, but whether the system stores both raw and

adjusted series or adjusts on read is unresolved; it affects how returns (FR-MKT-003) are defined and, because the adjusted series is published as events, every downstream read model inherits the choice.

Natural-language explanation uses an external LLM API. FR-XAI-005 assumes a hosted provider, which is why NFR-SEC-003 (secret management) and the cached-fallback clause exist. Choosing a local open model instead would change latency, cost, and the secret story.

Some breakdowns depend on reference data that may be incomplete. Sector and geography breakdowns (FR-RSK-008) and factor-level risk decomposition (FR-XAI-003) require per-instrument classification and factor-exposure data; they are Extended precisely because that data is not guaranteed for every instrument.

"Normal load" is not yet quantified. The performance NFRs (§3.1) reference "normal load"; concrete numbers — instruments tracked, concurrent users, maximum N×T per simulation, hot-data retention horizon — should be fixed once sizing is agreed, so the thresholds become fully testable.

GPU capacity bounds concurrent ML work. The ML worker deployment assumes a shared, bounded GPU budget; the number of permitted concurrent trainings and the demo hardware need to be pinned down.

The convergence bound in NFR-INT-007 needs a number. Read models must converge "within a stated bound", and the bound is not yet stated. It should be set per projection rather than globally, because a dashboard tile tolerating five seconds of lag and a risk-breach alert tolerating five seconds are not the same claim — and until a number exists, NFR-INT-007 is only half-testable.

Local resource budget for eight services. Every Core NFR is asserted to be demonstrable on the local Compose stack (§0), which must run eight service APIs, seven worker deployments, the gateway, PostgreSQL, Kafka, the schema registry, Redis, and MinIO on one machine (ARC §10.1, §15). If the team's hardware cannot hold that, the documented fallback is a Compose profile that starts a subset of services per demonstration — not a change to the architecture, and not a quiet downgrade of a Core requirement to Extended.

Contract-test tooling is unchosen. NFR-MNT-005 binds the team to running consumer-driven contract tests in each provider's pipeline; whether that is Pact, schemathesis against each service's OpenAPI document, or generated client fixtures is open (ARC §15) and determines how much of that requirement is automated versus manual.

6. Summary

This specification defines **81 functional requirements** across a cross-cutting Platform/Identity group and the proposal's seven modules, and **50 active non-functional requirements** across ten quality categories (51 IDs issued, one withdrawn at v0.2), each written to be individually testable and each traced to the architecture section that realizes it. The functional set is prioritized so that the platform's central premise — a simulation-driven, event-sourced, explainable, topology-aware risk-analytics platform running end-to-end — is entirely within the **Core** tier and demonstrable on a single local stack, while breadth (additional stochastic models, stress scenarios, predictive tasks, richer breakdowns) and production-grade operational qualities (replica failover, sub-five-minute PITR, HA broker, independent per-service restore) are **Extended**: designed and coded-for, delivered as time allows.

The functional set grew from 77 at v0.2's first draft to 81 in this revision, and the additions came from auditing this document against the proposal rather than from new ideas. Data cleaning and normalization (FR-MKT-011) and risk-concentration metrics (FR-RSK-018) are capabilities the proposal names explicitly that

no requirement had actually stated. Strategy comparison was separated into FR-DSH-008 so that risk heatmaps and volatility analysis could be raised to Core, where the availability of their inputs already put them. Alert deduplication (FR-DSH-009) was promoted from an implementation detail to a requirement because "one notification per condition" is a property users judge the feature by, and at-least-once delivery makes repeat notifications certain unless it is specified. Two further corrections belong to the same pass: FR-MKT-004 now owns the rolling correlation series that FR-RSK-011 consumes, resolving a duplicated computation, and every claim of coverage in §4.1 now points at a requirement that states the capability rather than one that merely touches it.

Otherwise the functional requirements are unchanged from version 0.1, because *what* the platform must do does not depend on how it is decomposed. The non-functional set grew from 34 to 50 because *how well* it must do it does: choosing microservices (ARC §0 A1) commits the team to graceful degradation when one of eight services is unavailable, atomic commit-and-publish, duplicate-safe event handling, saga compensation, cross-service referential soundness without foreign keys, bounded read-model convergence, per-hop trace propagation, independent deployability, and additive contract evolution. Those sixteen additional requirements are the accepted cost of the architectural decision, written as acceptance gates rather than left as good intentions — which is the point of specifying them here instead of discovering them during integration.

Two coverage claims close the document. The proposal matrix (§4.1) shows every capability in the approved seven-module proposal is carried by at least one requirement, and lists separately the requirements that go beyond it, so an addition is never mistaken for a proposal obligation. The owning-service register (§4.2) fixes which service owns which data, records the two places where the team brainstorm's grouping was rejected in favour of the proposal, and is binding rather than descriptive: it is the register every no-shared-data rule in §3 is checked against.